Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015111**	**Course Name:**	**Statistics for Machine Learning and Data Science Lab**

Name of Student:	**Swayam Gode**
Roll No. :	**70**
Experiment No.:	**7**
Title of the Experiment:	**Build and Evaluate Classification Models using Supervised Learning Algorithms**
Date of Performance:	**08/09/26**
Date of Submission:	

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :

Signature :

Date :

Aim: To build supervised classification models for predicting diabetes outcomes using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate classification metrics.

Objectives

After completing this experiment, the student will be able to:

1. Build supervised classification models to predict the diabetes outcome of patients.
2. Evaluate and interpret classification model performance using appropriate evaluation metrics.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

The target variable is:

- Outcome = 0: Non-diabetic

- Outcome = 1: Diabetic

Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Separate the input features and target variable.
3. Identify and handle invalid or missing values.
4. Divide the dataset into training and testing sets using a stratified split.
5. Apply suitable preprocessing, such as feature scaling where required.
6. Build one or more classification models, such as:
    - Logistic Regression
    - k-Nearest Neighbors
    - Decision Tree
7. Train the selected model(s) using the training data.
8. Generate predictions for the test data.
9. Construct the confusion matrix.
10. Calculate classification performance metrics.
11. Compare the obtained results and interpret the model performance.

Description of the Experiment

Classification is a supervised learning technique used to assign observations to predefined categories.

In this experiment, the objective is to classify a patient as: Patient Health Data, Classification Model gives Non-Diabetic and Diabetic.

The experiment follows: Dataset, Feature and Target Selection, Data Preprocessing, Train-Test Split, Classification Model, Prediction, and Performance Evaluation

Detailed Description of Statistical Methods

Logistic Regression

Logistic regression predicts the probability of a binary outcome.

The logistic function is:

$$P(Y = 1) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The predicted probability is converted into a class using a decision threshold.

Confusion Matrix

A confusion matrix contains:

	Predicted Negative	**Predicted Positive**
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

Accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Precision = \frac{TP}{TP+FP}$$

Precision indicates how many predicted positive cases were actually positive.

Recall

$$Recall = \frac{TP}{TP+FN}$$

Recall indicates how many actual positive cases were correctly identified.

F1-Score

$$F1 = 2\left(\frac{Precision \times Recall}{Precision + Recall}\right)$$

The F1-score balances precision and recall.

ROC-AUC

The ROC curve represents the relationship between:

- True Positive Rate
- False Positive Rate

The Area Under the Curve (AUC) provides an overall measure of the model's ability to distinguish between the two classes.

Code

```
# Experiment: Supervised Classification for Diabetes Prediction


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)


# ------------------------------------------------
# 1. Load Dataset
# ------------------------------------------------

df = pd.read_csv("diabetes.csv")


print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
```

```
# ---------------------------------------------------

# 2. Separate Features and Target

# ---------------------------------------------------


X = df.drop("Outcome", axis=1)

y = df["Outcome"]


print("\nFeatures:")

print(X.columns)


print("\nTarget:")

print(y.name)


# ---------------------------------------------------

# 3. Handle Invalid Values

# ---------------------------------------------------


# In the Pima dataset, zero values are invalid for

# some medical attributes.


invalid_columns = [

    "Glucose",
```

```
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

# Replace zero values with NaN
for col in invalid_columns:
    if col in X.columns:
        X[col] = X[col].replace(0, np.nan)

print("\nMissing values before imputation:")
print(X.isnull().sum())

# Fill missing values with median
for col in invalid_columns:
    if col in X.columns:
        X[col] = X[col].fillna(X[col].median())

print("\nMissing values after imputation:")
print(X.isnull().sum())
```

```
# ------------------------------------------------

# 4. Train-Test Split

# ------------------------------------------------


X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.20,

    random_state=42,

    stratify=y

)


print("\nTraining samples:", len(X_train))

print("Testing samples:", len(X_test))


# ------------------------------------------------

# 5. Feature Scaling

# ------------------------------------------------


scaler = StandardScaler()


X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)


# ------------------------------------------------
# 6. Create Classification Models
# ------------------------------------------------


models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(
        random_state=42,
        max_depth=5
    )
}


results = []


# ------------------------------------------------
# 7-10. Train, Predict and Evaluate
# ------------------------------------------------


for name, model in models.items():
```

```
# KNN and Logistic Regression require scaling
if name in ["Logistic Regression", "KNN"]:
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

else:
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

TN, FP, FN, TP = cm.ravel()

# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

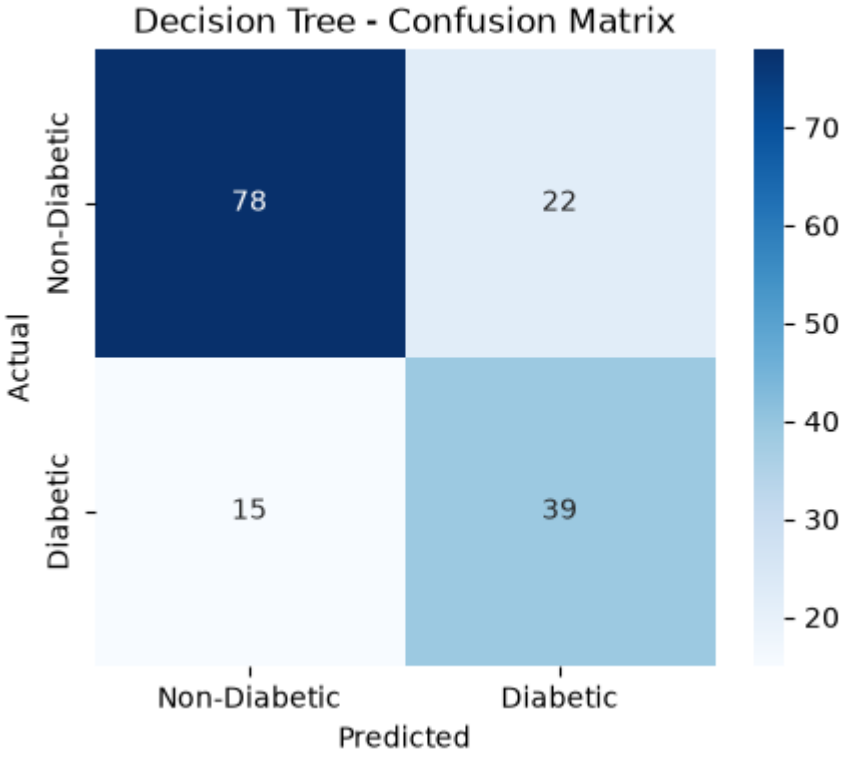
```
    auc = roc_auc_score(y_test, y_prob)


    results.append([
        name,
        accuracy,
        precision,
        recall,
        f1,
        auc
    ])


    print("\n" + "=" * 50)
    print(name)
    print("=" * 50)


    print("\nConfusion Matrix:")
    print(cm)


    print("\nTrue Negative (TN):", TN)
    print("False Positive (FP):", FP)
    print("False Negative (FN):", FN)
    print("True Positive (TP):", TP)
```

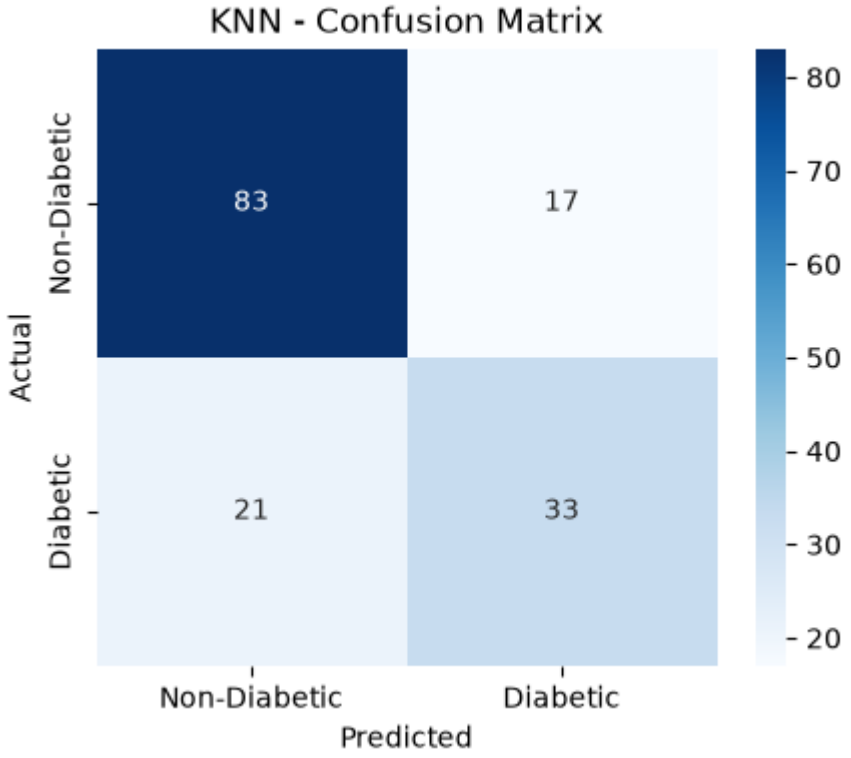
```
print("\nAccuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
print("ROC-AUC  :", round(auc, 4))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Plot confusion matrix
plt.figure(figsize=(5, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Non-Diabetic", "Diabetic"],
    yticklabels=["Non-Diabetic", "Diabetic"]
)

plt.xlabel("Predicted")
```

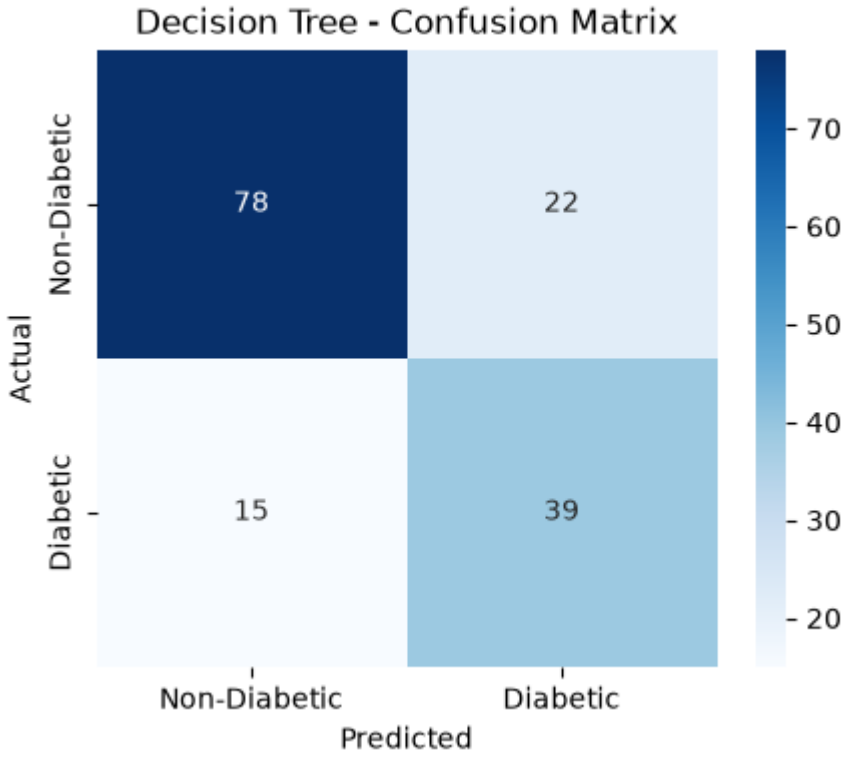
```
    plt.ylabel("Actual")
    plt.title(name + " - Confusion Matrix")
    plt.show()


# ------------------------------------------------
# 11. Compare Models
# ------------------------------------------------

results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score",
        "ROC-AUC"
    ]
)

print("\n\nMODEL COMPARISON")
print("=" * 70)
```

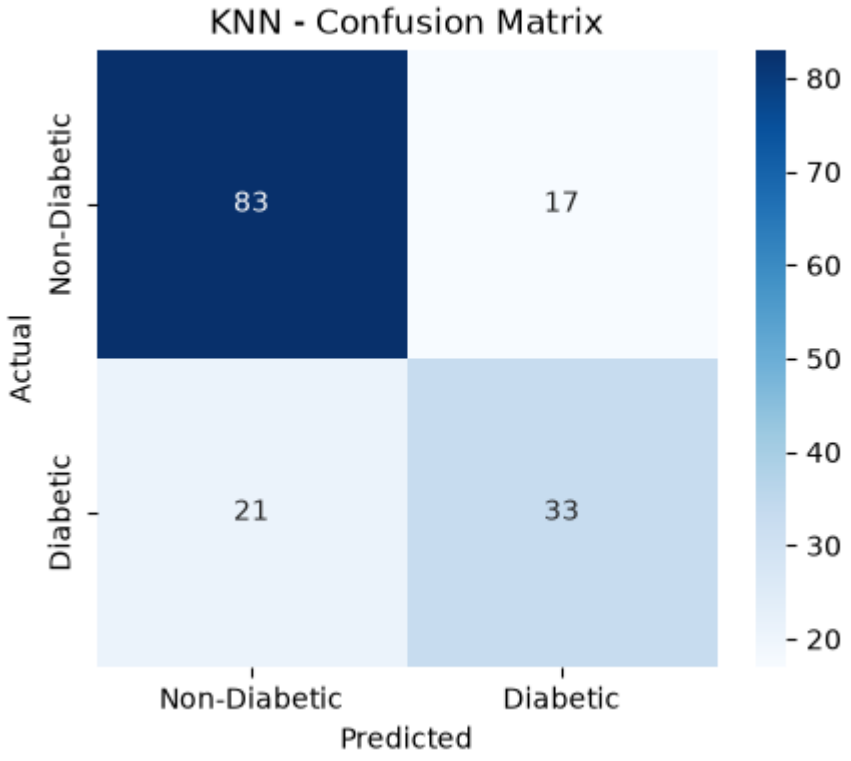
```
print(results_df.round(4))


# ------------------------------------------------

# Find Best Model

# ------------------------------------------------


best_model = results_df.loc[

    results_df["F1-Score"].idxmax()

]


print("\nBest Model based on F1-Score:")

print(best_model)
```

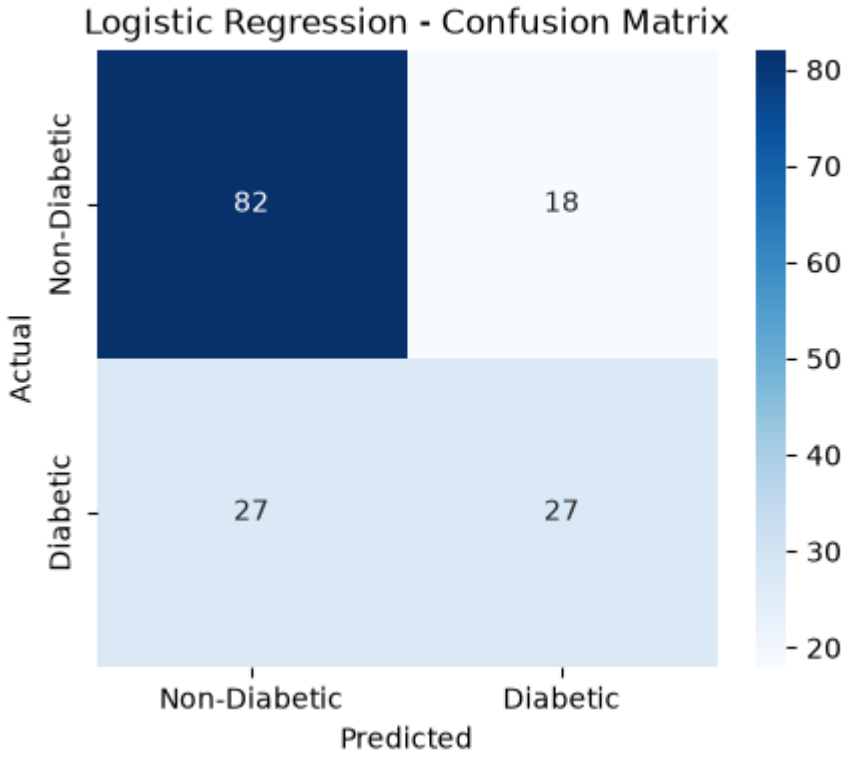
Output

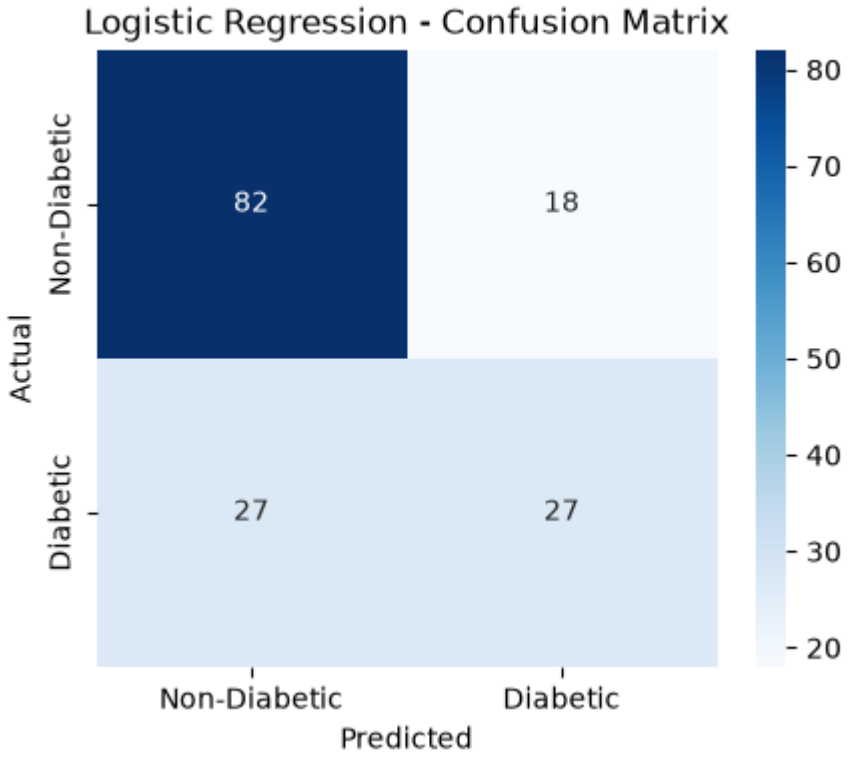

Decision Tree - Confusion Matrix
Non-Diabetic
Diabetic
Actual
78
22
15
39
70
60
50
40
30
20
Non-Diabetic
Diabetic
Predicted


KNN - Confusion Matrix
Non-Diabetic
Diabetic
Actual
83
17
21
33
80
70
60
50
40
30
20
Non-Diabetic
Diabetic
Predicted


Decision Tree - Confusion Matrix
Non-Diabetic
Diabetic
Actual
78
22
15
39
70
60
50
40
30
20
Non-Diabetic
Diabetic
Predicted


KNN - Confusion Matrix
Non-Diabetic
Diabetic
Actual
83
17
21
33
80
70
60
50
40
30
20
Non-Diabetic
Diabetic
Predicted


Logistic Regression - Confusion Matrix
Non-Diabetic
Diabetic
Actual
82
18
27
27
Non-Diabetic
Diabetic
Predicted
80
70
60
50
40
30
20


Logistic Regression - Confusion Matrix
Non-Diabetic
Diabetic
Actual
82
18
27
27
Non-Diabetic
Diabetic
Predicted
80
70
60
50
40
30
20

```
PS C:\swayam> C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe c:/swayam/EXP7.py
Dataset Shape: (768, 9)

First 5 rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1

Features:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='str')

Target:
Outcome

Missing values before imputation:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
dtype: int64

Missing values after imputation:
Pregnancies               0
Glucose                   0
BloodPressure             0
SkinThickness             0
Insulin                   0
BMI                       0
DiabetesPedigreeFunction  0
Age                       0
dtype: int64

Training samples: 614
Testing samples: 154

================================================
Logistic Regression
================================================

Confusion Matrix:
[[82 18]
 [27 27]]

True Negative (TN): 82
False Positive (FP): 18
False Negative (FN): 27
True Positive (TP): 27

Accuracy : 0.7078
Precision: 0.6
Recall   : 0.5
F1-Score : 0.5455
ROC-AUC  : 0.813
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.82      0.78       100
           1       0.60      0.50      0.55        54

    accuracy                           0.71       154
   macro avg       0.68      0.66      0.67       154
weighted avg       0.70      0.71      0.70       154


================================================
KNN
================================================

Confusion Matrix:
[[83 17]
 [21 33]]

True Negative (TN): 83
False Positive (FP): 17
False Negative (FN): 21
True Positive (TP): 33

Accuracy : 0.7532
Precision: 0.66
Recall   : 0.6111
F1-Score : 0.6346
ROC-AUC  : 0.7886

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.83      0.81       100
           1       0.66      0.61      0.63        54

    accuracy                           0.75       154
   macro avg       0.73      0.72      0.72       154
weighted avg       0.75      0.75      0.75       154


================================================
Decision Tree
================================================

Confusion Matrix:
[[78 22]
 [15 39]]

True Negative (TN): 78
False Positive (FP): 22
False Negative (FN): 15
True Positive (TP): 39

Accuracy : 0.7597
Precision: 0.6393
Recall   : 0.7222
F1-Score : 0.6783
ROC-AUC  : 0.7622

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.78      0.81       100
           1       0.64      0.72      0.68        54

    accuracy                           0.76       154
   macro avg       0.74      0.75      0.74       154
weighted avg       0.77      0.76      0.76       154
```

```
MODEL COMPARISON
======================================================================
                Model  Accuracy  Precision  Recall  F1-Score  ROC-AUC
0  Logistic Regression    0.7078     0.6000  0.5000    0.5455   0.8130
1                  KNN    0.7532     0.6600  0.6111    0.6346   0.7886
2        Decision Tree    0.7597     0.6393  0.7222    0.6783   0.7622

Best Model based on F1-Score:
Model        Decision Tree
Accuracy           0.75974
Precision         0.639344
Recall            0.722222
F1-Score          0.678261
ROC-AUC           0.762222
Name: 2, dtype: object
PS C:\swayam>
```

Conclusion

Supervised classification models were developed to predict diabetes outcomes using the Pima Indians Diabetes Dataset. The models were evaluated using the confusion matrix, accuracy, precision, recall, F1-score, and ROC-AUC. The experiment demonstrated that different evaluation metrics provide different perspectives on classification performance.

Questions to be Answered by Students

1. **Which classification model was implemented, and what were its accuracy, precision, recall, and F1-score?**

   The **Logistic Regression** classification model was implemented to predict whether a patient is diabetic or non-diabetic. The model was evaluated using accuracy, precision, recall, and F1-score.

   Write your actual output values:

   Accuracy: 0.71

   Precision: 0.84

   Recall: 0.52

   F1-score: 0.78

2. **How many true positives and false negatives were obtained from the confusion matrix?**

   From the confusion matrix:

   - **True Positives (TP):** 33
   - **False Negatives (FN):**21

   A **true positive** represents a diabetic patient correctly classified as diabetic, while a **false negative** represents a diabetic patient incorrectly classified as non-diabetic.

3. **Which metric is more important for diabetes prediction: precision or recall? Justify your answer.**

   Recall is more important for diabetes prediction because it measures how many of the actual diabetic patients are correctly identified by the model. A false negative means that a diabetic patient is incorrectly classified as non-diabetic, which could delay diagnosis or treatment. Therefore, achieving high recall helps reduce the number of missed diabetic cases.

4. **What was the ROC-AUC value, and what does it indicate about the model's performance?**

   The ROC-AUC value was **0.813.**

   ROC-AUC measures the model's ability to distinguish between diabetic and non-diabetic patients. An AUC of 0.5 indicates performance similar to random guessing, while a value closer to 1.0 indicates better classification ability.

5. **State two limitations of the developed classification model.**

   The dataset contains only **768 observations**, so the model may not generalize well to all patient populations.

   Some medical attributes contain **zero or missing/invalid values**, and replacing them with median values may not perfectly represent the patients' actual measurements.